

Planet Wars Competition Entry

Planet Warrior V2

Planet Warrior Evolutionary Agent for fully observable Planet Wars
Planet Warriors: Melih Ustun, Ben Pittman, Nikolay Popov, Deniz Atar
Queen Mary University of London

Agent Overview

- Goal: choose one strong real-time action plan every game tick under a set time budget through a Rolling Horizon evolutionary algorithm.
- Represents the plan as a sequence of triples: source index, target index, and ship fraction.
- Starts from a logical sequence sampled from another heuristic agent that was developed (DefensiveReactiveAgent11) instead of random noise.
- Evolves the plan with through mutations and keeps the best scored sequence at every mutation cycle.
- Final action for the tick is the first triple of the best sequence.

System Design

1. Seed: simulate DefensiveReactiveAgent11 to sample an initial action sequence.
2. Evaluate: run a Forward Model against an opponent model (DoNothingAgent in this case).
3. Score: ship advantage + weighted growth-rate advantage.
4. Improve: mutate and evaluate possible sequences until the tick time limit expires.
5. Action: Return the best action found when the time limit expires.
6. Reuse: shift the winning sequence left, append random future actions.
7. Refresh: every 80 ticks, compare against a new defensive rollout seed.

Results

Benchmark test results (100 games per test):

- vs PureRandomAgent: 100%
- vs BetterRandomAgent: 100%
- vs CarefulRandomAgent: 100%
- vs GreedyHeuristic: 97%
- vs SimpleEvoAgent: 100%

Test config: 20 planets, 2000 ticks, growth rate: 0.05-0.2, transporter speed: 3.5

Analysis and Insights

- Main improvement: initializing (and reinitializing when advantageous) search from a competent heuristic policy, not a random vector helping the model not get stuck in local maxima while still converging.
- Growth-rate scoring rewards economy; the agent values future production as well as ships, the weighting of which changes over the course of the game.
- Shift buffer provides continuity between ticks and avoids replanning from scratch.
- Explicit doNothing handling prevents tiny or invalid launches from wasting transporter slots, as well as allowing for doNothing to be actions to be mapped into the sequence.
- Risk: forward-model search is expensive.
- Risk: opponent model kept at DoNothingAgent, so adversarial prediction remains shallow.

Future Work

- Tune sequence length, mutation probability, economy weights, and the 80-tick reseed interval to find the most optimal values.
- Incorporate and test a second sequence seeded from a different powerful model for broader strategic diversity.
- Conduct an ablation study to find the impacts of different seeding strategies and optimize them: random seed vs sampled seed, periodic reseeding vs no periodic reseeding, etc.
- Use machine learning to test and optimize some of the parameters over many epochs.
- Port the approach to partial observability with belief-state reconstruction.